

IICT · INNOVATIONS IN COMPILER TECHNOLOGY

Waspy

Direct ahead-of-time compilation of typed Python
to compact WebAssembly

Kumar Anirudha & Farhaan Bukhsh

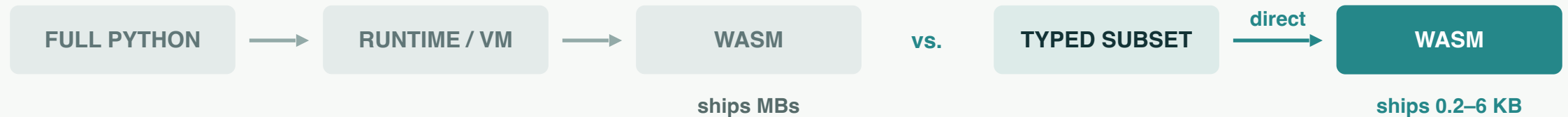
github.com/anistark/waspy



THE QUESTION

Do we need to ship Python to run Python?

Interpreter-in-Wasm gives compatibility. Waspy asks what happens when we compile the program instead of the language runtime.



How much Python can we compile without carrying a general Python runtime?

Three different points in the design space

Interpreter in Wasm

Pyodide / CPython builds

Goal: **preserve Python semantics and ecosystem.**

Ships: **CPython + stdlib (+ packages).**

Trade-off: **MB-scale fixed runtime cost.**

Compile + Python runtime

py2wasm / Nuitka-like approaches

Goal: **compile ahead of time while retaining runtime semantics.**

Ships: **generated code + Python runtime.**

Waspy

Goal: **compile a closed-world typed subset directly to Wasm.**

Ships: **reachable user code + compiler intrinsics.**

Trade-off: **narrower semantics; KB-scale output.**

**Optimize for deployment,
not completeness**



RELATED SYSTEMS

The design space for Python on Wasm

	CPython in Wasm	py2wasm	MicroPython	SPy	Waspy
Strategy	CPython interpreter	Python → C, links CPython	MicroPython VM	Python variant, redshift → C	Direct AOT to Wasm
Target	Wasm	Wasm	Wasm	native + Wasm	Wasm
Runtime shipped	CPython + stdlib	CPython runtime	MicroPython VM	libspy + Boehm GC	none
Wasm payload	MBs (> 6 MB)	MBs	100s of KB	not reported	0.2 – 6 KB
Coverage	effectively full	near-full	large subset	Python variant	typed subset
Host interface	eval-style entry	via embedded runtime	eval-style entry	via C / Emscripten	typed Wasm exports
Execution	interpreted	~3x vs interpreted	interpreted (VM)	compiled via C	compiled; not yet benchmarked
Memory	runtime-managed	runtime-managed	runtime-managed	Boehm GC	bump alloc, no free yet

THE DESIGN POINT

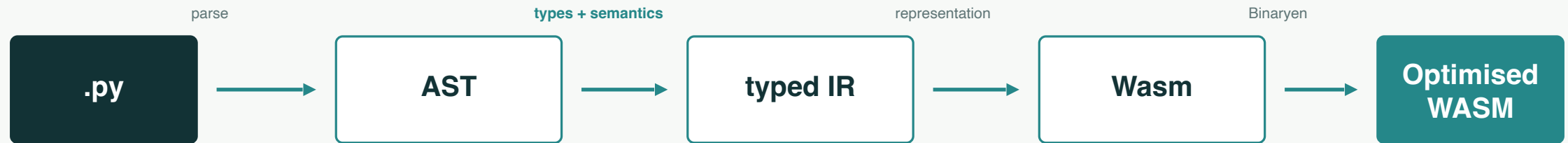
Ship a program, not a language implementation.

Closed-world AOT: only user code and the stdlib intrinsics it reaches are emitted. Exports carry Wasm function types.

“The gap is structural, not incidental.”

ARCHITECTURE

The important step is lowering semantics



Parse

rustpython-parser

CPython grammar →
syntax tree

Lower

Resolve types, symbols, control
flow and representation choices.

This is where dynamic choices
become static.

Codegen

wasm-encoder

Emit binary Wasm directly
— no C/LLVM/text round-
trip.

Optimise

Binaryen

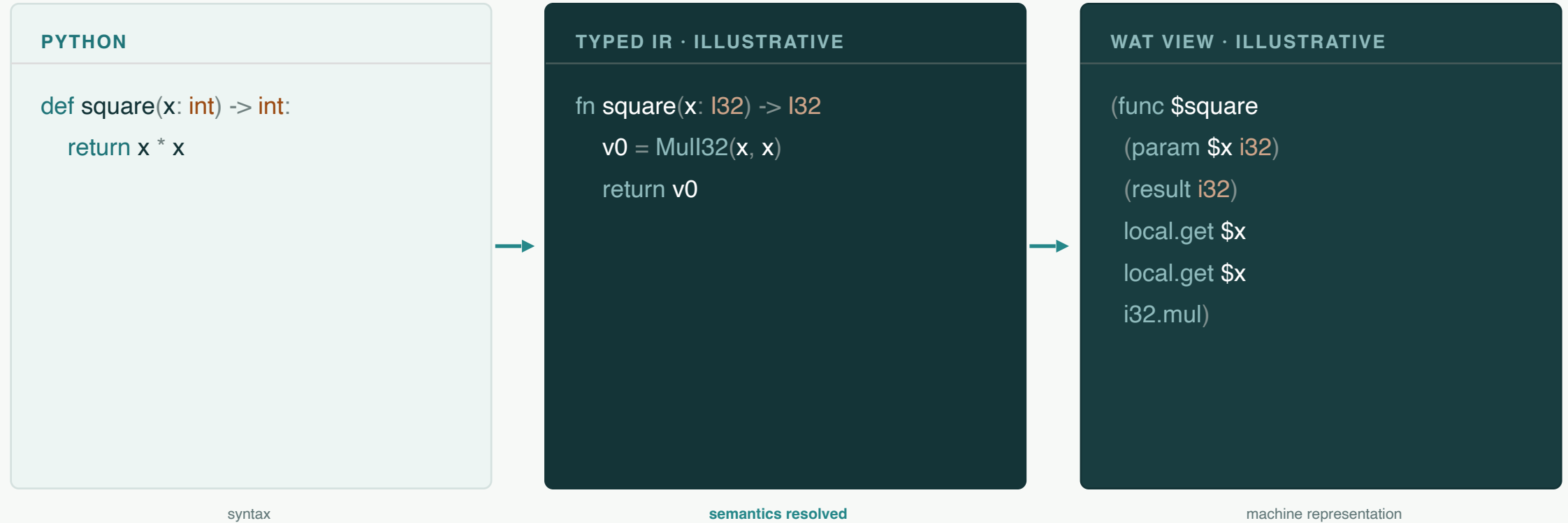
DCE, inlining, local
simplification and size
reduction.

Once we reach the typed IR, code generation no longer needs to understand Python.

Demo

WORKED EXAMPLE

One function, three compiler views



SEMANTICS

CPython vs Waspy

Construct	CPython	Waspy	Status
float	IEEE-754 double	f64, IEEE-754	same
% and //	sign follows divisor	Python sign semantics kept	same
and / or	short-circuit	conditional branches	same
int	arbitrary precision	fixed-width i32	differs
list growth	dynamic resize	capacity fixed from literal	differs
object lifetime	refcount + GC, __del__ runs	bump allocator, never freed	differs
generators	suspend and resume	yield keeps no state	differs
closures	full capture	lambdas, basic closures	partial



Differential testing: input is valid CPython, so each feature is checked by running the same source under CPython and as Wasm, and comparing results.

How Python values land in Wasm

Python	Wasm	Representation
int	i32	32-bit two's complement; wraps on overflow
float	f64	IEEE-754 double
bool	i32	0 or 1
str	i32	Offset into linear memory; 4-byte length prefix
list / tuple	i32	Pointer to header: len · cap · data ptr; 8-byte slots
dict	i32	Pointer; each entry is a key slot + value slot
set	i32	Pointer to an open-addressing hash table
class instance	i32	Pointer to field block; class tag at offset 0

EXAMPLE · SIGNATURE & LAYOUT

```
def area(w: int, h: float) -> float:
    return w * h
```

```
(func $area (param i32 f64)
  (result f64))
```

```
xs = [1.5, 2.0]
# header len=2 cap=2 data->
# data 1.5:f64 2.0:f64
```

```
2147483647 + 1 # -2147483648
```

LIMITATIONS

Limitations of the current subset

No memory reclamation

The bump allocator has no free. Instances live until teardown; `__del__` never runs.

Fixed-capacity collections

Sized at compile time from literals, so an unbounded `.append()` is unsafe.

Generators

`yield` doesn't preserve execution state yet.

Closures

Capture analysis is incomplete. Lambdas and basic closures work.

Imports

Only the bundled `stdlib`. Your own modules merge at the project level.

File I/O

Waiting on a WASI-backed implementation.

Also pending: `classmethod` / `staticmethod`, `dataclasses`, hashed dict lookups (sets already hash), module caching.

GET INVOLVED

Contributors and researchers welcome

REPOSITORY

github.com/anistark/waspy

MIT · Rust

 cargo add waspy

For contributors

- Memory reclamation: refcounting or Wasm GC
- Growable collections, hashed dict lookups
- Generators, full closure capture, dataclasses
- More stdlib modules, WASI-backed file I/O

For researchers

- Memory management for KB-scale modules
- Execution-time evaluation vs interpreter-in-Wasm
- CPython as a differential-testing oracle
- Typed IR and embeddable crate to build on

THANK YOU

Questions?

Kumar Anirudha

x.com/kranirudha

fosstodon.org/@ani

Farhaan Bukhsh

x.com/fhackdroid

mastodon.social/@fhackdroid



github.com/anistark/waspy

anistark.github.io/waspy